Assignment: Tomasulo Algorithm Implementation
COSC 6385 Computer Architecture
Daniel Prol - UH ID: 2446940
Repository: https://github.com/dprol/tomasulo-algorithm

Introduction

Tomasulo's algorithm schedules instructions dynamically. Instructions are issued in program order, start execution as soon as their operands are ready, even if earlier instructions are still waiting, and commit in program order through the reorder buffer. Register renaming removes the false dependencies between them.

In this assignment I have done the following:

1. Modified the existing Python implementation of the Tomasulo algorithm to issue 1 to 4 instructions per cycle, in program order. With issue width n, up to n results are broadcast on the CDB and up to n instructions commit per cycle, so that write back and commit do not limit a wider issue. These are the assumptions of the multi-issue example of the course lecture slides (CompArch-OOO-Example.pdf).
2. Added an input file with the hardware configuration, the initial register and memory values, the program and the issue width (Issue width = n, default 1). The issue width can also be given on the command line.
3. Fixed several problems of the provided code that gave wrong values or wrong timing (listed in "Problems of the provided code that I fixed").
4. Tested the simulator with 6 test cases and issue width 1 to 4, and checked every run against a sequential reference model (a simple interpreter that executes one instruction at a time, with no pipeline).

What works

- Issue, write back and commit of up to n instructions per cycle, for issue width 1 to 4.
- The output has the timing table and the final register and non-zero memory values.
- As a check of the timing, I ran the 6-instruction example that is worked cycle by cycle in the course lecture slides (CompArch-OOO-Example.pdf: two loads, two FP multiplies, one FP add and one integer add). The simulator gives exactly the same timing table as the slides, with one issue per cycle (40 cycles, slide 35) and with two issues per cycle (39 cycles, slide 63). Both tables are in the test cases section (test6_class_example).
- All 6 test cases give the same final registers and memory as the reference model for every issue width (24 runs).

Limitations

- There is no branch prediction. The provided code stalls fetch after a branch until the branch is resolved in EX, and I kept that. With multiple issue, no instruction after a branch is issued in the same cycle. The 1-bit predictor and the BTB of the

assignment text are not implemented, so there is no speculation and no squashing.
- Only 1 functional unit per type is supported (a different # of FUs gives a warning and 1 is used).
- The branch offset is in bytes, as in the provided code and its test case (Bne R1 R0 -12 goes 3 instructions back), not in words as in the assignment text.

Submitted files

The code is submitted in tomasulo-algorithm.zip, together with this report. The same files are in the repository.

Path	Content
code/	The simulator. main.py has the main loop, init.py has the data types and the input file reader, and there is one file per stage: issue.py, exe.py, mem.py, wb.py and commit.py. run_tests.py runs and checks all the tests.
tests/	The 6 test cases (input files).
results/	The output of every test with issue width 1 to 4 (24 files) and summary.txt.
changes.diff	Every line that I changed in the provided code, as a unified diff.
README.md	How to run the simulator and the format of the input file.

This report does not copy the code. Each change is described in words, with the file, the function and the lines where it is. The line numbers are those of the submitted files.

Code Changes

The provided code issues, broadcasts and commits one instruction per cycle, and its configuration is written in main.py. My changes are in four groups. changes.diff shows the provided version and the new version of every changed line.

Multiple issue (issue.py, main.py)

The provided code issues at most one instruction per cycle. I added an issue_width parameter (1 to 4): the number of instructions that can be issued in the same cycle.

Change	Where
The provided issue function is now issue_one, and it returns whether the instruction was issued	issue.py, issue_one (lines 176–258)
New issue function: it calls issue_one up to issue_width times per cycle	issue.py, issue (lines 262–279)
The issue width is read from the command line or from the input file, and checked (1 to 4)	main.py, lines 20–25
The issue width is passed to issue every cycle	main.py, lines 110–117

issue_one has the same body as the provided function. The only addition is a flag that is set in the five branches where the instruction enters the ROB (load/store, integer, FP add, FP multiply and branch) and is returned at the end. It stays false when the ROB, the reservation station or the load/store queue has no space.

The new issue is a loop of at most issue_width iterations. It stops when there is nothing left to fetch or fetch is stalled by an unresolved branch, and it stops when issue_one could not issue the instruction.

Three things keep the issue in program order:

- The loop stops at the first instruction that cannot be issued (ROB, reservation station or load/store queue full), so a later instruction never passes it.
- The RAT is updated after each instruction, so an instruction that depends on another one issued in the same cycle gets the ROB tag of that instruction. For example, when Mult.d F1 F2 F4 and Mult.d F8 F6 F1 are issued in the same cycle, the second one reads F1 after the first one has renamed it, and waits for the result of the first multiply instead of taking the old value of F1.
- The loop stops after a branch, because there is no branch prediction and the next PC is not known until the branch is resolved.

Write back and commit (wb.py, commit.py, main.py)

The provided code broadcasts one result per cycle on the CDB and commits one instruction per cycle. With issue width n, my version broadcasts up to n results and commits up to n instructions per cycle, so that write back and commit do not limit a wider issue. These are the assumptions of the multi-issue example of the course lecture slides (CompArch-OOO-Example.pdf). With issue width 1 there is one broadcast per cycle, as in the provided code.

Change	Where
The CDB is a list of issue_width entries instead of one entry	main.py, lines 80–87
wb broadcasts and loads every entry of the list	wb.py, wb (lines 61–79)
commit is rewritten as a loop over the head of the ROB	commit.py, commit (lines 30–52)
The issue width is passed to commit, and the cycle of the last commit is kept for the total	main.py, lines 137–141

In the provided code, wb broadcasts the result that was put on the CDB in the previous cycle, and then puts the next result of the queue on the CDB. My version does the same two steps for every entry of the list. A result still waits one cycle between the end of EX (or MEM) and its broadcast. If more than n results finish in the same cycle, the rest wait in results_buffer in the order in which they finished.

In the provided code, commit removes at most one instruction with a result per cycle, and it removes a branch from the ROB without a commit cycle. In my version:

- The loop always looks at the head of the ROB and stops at the first instruction that is not finished, so the commit is in program order.
- A counter limits the commits of one cycle to issue_width.
- A branch commits when it has been resolved in EX. It now gets a commit cycle and uses one of the n commit slots.
- A store commits when it starts writing memory. The MEM stage writes its commit cycle, and only when the store is at the head of the ROB (see "Problems of the provided code that I fixed"). commit only removes it from the ROB.
- Any other instruction commits the cycle after its broadcast, and its value is written to the architectural register.

Input and output (init.py, main.py, issue.py, exe.py, commit.py)

In the provided code the program was read from code.in, and the hardware configuration and the initial values were written in main.py. My version reads everything from one input file in the format of the assignment text, with one new line for the issue width. This is the beginning of tests/test3_forwarding.txt. The file is plain text; the first block is the configuration of the units:

Unit	# of rs	Cycles in EX	Cycles in Mem	# of FUs
Integer adder	2	1		1
FP adder	3	3		1
FP multiplier	2	20		1
Load/store unit	3	1	4	1

The rest of the file has one item per line:

Line in the file	Meaning
ROB entries = 128	Number of ROB entries.
Issue width = 2	Issue width, 1 to 4. This is the new line; without it the issue width is 1.
R1=10, R2=20, F2=30.1	Initial register values. The other registers are 0.
Mem[4]=1, Mem[8]=2, Mem[12]=3.4, Mem[18]=7.5	Initial memory values, with byte addresses.
Add.d F1, F2, F3 Ld F4, 8(R1) ...	The program, one instruction per line.

The simulator is run as python3 main.py [input file] [issue width]. The default input file is test_case.txt, and an issue width on the command line replaces the one of the file.

Change	Where
New input file reader	init.py, read_input (lines 74–112), with to_number (lines 64–68) and normalize_instruction (lines 70–72)
Registers, memory, sizes and latencies come from the input file	main.py, lines 26–79
Register numbers with two digits in Ld/Sd	issue.py, put_ins_into_ldsd (lines 145–147)
Beq	issue.py, put_ins_into_ROB (line 38) and issue_one (lines 244–255); exe.py, fu_exe (lines 81–83); commit.py, commit (line 34)
R0 is hardwired to 0	main.py, lines 31–33; issue.py, update_rat (lines 168–169); commit.py, modify_arch_reg (lines 21–22)
Output: issue width, total cycles, registers and non-zero memory	main.py, line 97 and lines 148–168

- Input file. read_input returns a dictionary with the configuration, the initial values and the instructions. In the provided code these values were written in main.py, with the values of the provided test case. Instructions are case insensitive, and their operands can be separated with spaces or commas.

- Register numbers with two digits. The provided code took the register name of a load or store from fixed character positions, so 0(R1) worked but 0(R10) did not. Now the operand is parsed with a regular expression.
- Beq. The provided code had only Bne. Beq is issued in the same way, its offset is stored in the ROB in the same way, and it is taken when the two values are equal.
- R0. An instruction with destination R0 does not rename R0 and does not write it at commit. An initial value for R0 in the input file is ignored.
- Output. The provided code printed only the timing table. My version prints the issue width before the table, and after it the total number of cycles, all the registers and the non-zero memory values.

Problems of the provided code that I fixed

Some of these problems were hidden with one issue per cycle and appeared with multiple issue. The others I found by comparing the results with the sequential reference model (see "Results and Test cases").

Where	Problem	Fix
wb.py, boroadcast (lines 27–35)	A broadcast always wrote its value in the RAT, also when a later instruction had already renamed the register. Later instructions then read an old value.	The RAT entry is updated only if it still has the tag of the broadcast.
exe.py, check_valid_ins_in_rs (lines 24–33), check_valid_ins_in_ldsd (lines 36–44) and exe (lines 137–170)	An instruction issued in a cycle was moved to the functional unit in that same cycle, so with 2 issues per cycle two instructions used the single integer adder (or address adder) in the same cycle.	Only instructions issued in a previous cycle are selected, the oldest first. The integer adder takes a new instruction only when it is free (unpipelined).
exe.py, fu_exe (lines 57–63)	When an instruction entered a pipelined unit, all the instructions already in the unit advanced a second time in that cycle and finished early.	Only the new instruction advances.
exe.py, ld_sd_execution (lines 106–107)	The address calculation never finished with more than 1 cycle in EX.	The cycle counter increases every cycle.
mem.py, check_if_sd_committable (lines 63–64), called in mem (line 125)	check_if_sd_committable was never called (the if tested the function name), so a store could write memory before older instructions committed.	A store goes to memory only when it is at the head of the ROB.
mem.py, mem (lines 70–86)	The forwarding loop reused its index variable, and a load could be forwarded in its EX cycle.	Separate variable; forwarding only after the EX cycle.

mem.py, mem (lines 136–144)	A load could not go to memory while a store with data was at the head of the queue, also with different addresses.	The check was removed; check_all_previous_Sd_no_match already does the disambiguation.
main.py, lines 108 and 145–146	The simulation ended when the ROB was empty, also if a branch had just been resolved and instructions were left, or if the last store was still writing.	The loop also continues while there are instructions to fetch or the memory unit is busy.

With issue width 1 the timing of the provided test case is the same as with the provided code, with one exception: the three branches now have a commit cycle (35, 38, 41), and the instructions after them commit one cycle later for each branch (the last commit is in cycle 42 instead of 39).

Results and Test cases

I wrote 5 test cases in addition to the provided one. Each test case is one input file in tests/, with the hardware configuration, the initial values and the program. Every test was run with issue width 1, 2, 3 and 4 (24 runs). The complete output of every run is in results/, in the submitted code and in the repository.

How the results are checked. code/run_tests.py runs all the tests and checks every run automatically:

- The committed instructions, the final registers and the final memory are compared with a sequential reference model: a simple interpreter that executes the same program one instruction at a time, with no pipeline. Out-of-order execution and multiple issue must not change the result of the program.
- At most issue width instructions are issued, broadcast and committed per cycle, and issue and commit are in program order.
- Every instruction goes through ISSUE, EX, MEM, WB and COMMIT in increasing cycles, with the configured number of cycles in EX and MEM.
- Only one instruction at a time uses the integer adder, the address adder and the memory port, and stores write memory at commit.
- For test6_class_example, the complete timing table is compared with the tables of the lecture slides (issue width 1 and 2).

Test	What it tests
test1_loop	The provided test case (code/test_case.txt): a loop with a Bne.
test2_independent	12 instructions with few dependencies and no branches.

test3_forwarding	The sample configuration of the assignment text, with comma syntax. Store-to-load forwarding, loads that pass stores with a different address, and a load/store queue of 3 entries that fills.
test4_small_structures	ROB of 4 entries and 1 reservation station per unit, 2 cycles in the integer adder, a register renamed twice (F7), and a write to R0. Structural hazards stop the issue in the middle of a group.
test5_branches	Lower case input. A loop that runs 5 times with load, multiply, add, store and load of the same address (Bne taken 4 times and not taken once), a Beq that is not taken and a Beq that is taken and skips one instruction.
test6_class_example	The 6-instruction example of the course lecture slides (CompArch-OOO-Example.pdf), with the latencies of the slides.

Total cycles for each issue width

Test	Width 1	Width 2	Width 3	Width 4	Result
test1_loop	42	36	35	35	pass
test2_independent	21	17	17	16	pass
test3_forwarding	50	48	47	47	pass
test4_small_structures	48	46	46	46	pass
test5_branches	69	72	70	69	pass
test6_class_example	40	39	38	38	pass

The final registers and memory are the same for the 4 widths in every test, and equal to the reference model.

test2_independent has the largest gain (21 to 16 cycles) because its instructions have few dependencies. In the other tests the gain is small because the rest of the machine is the same: one unit of each type, one memory port, and fetch stalled after every branch. The instructions enter the ROB earlier, but then they wait for a long multiply, for

the memory port or for a reservation station. test5_branches is slower with width 2 and 3 than with width 1; the reason is explained in that test case.

In the tables, each column has the cycles of one stage: first–last for EX and MEM (a forwarded load has a single MEM cycle), and one cycle for ISSUE, WB and COMMIT. Instructions are listed in commit order.

Test case 1: test1_loop (provided test case)

The provided test case, a loop with a Bne, takes 42 cycles with issue width 1 and 36 with issue width 2.

Configuration (ROB entries: 64)

Unit	Reservation stations	Cycles in EX	Cycles in MEM
Integer adder	4	1	
FP adder	3	4	
FP multiplier	2	15	
Load/store unit	5	1	5

Initial values: R1=12, R2=32, F20=3.0; Mem[4]=3.0, Mem[8]=2.0, Mem[12]=1.0, Mem[24]=6.0, Mem[28]=5.0, Mem[32]=4.0

Program

```
Ld F2 0(R1)
Mult.d F4 F2 F20
Ld F6 0(R2)
Add.d F6 F4 F6
Sd F6 0(R2)
Addi R1 R1 -4
Addi R2 R2 -4
Bne R1 R0 -12
Add.d F20 F2 F2
```

Issue width 1 (42 cycles)

Instruction	ISSUE	EX	MEM	WB	COMMIT
Ld F2 0(R1)	1	2–2	3–7	9	10
Mult.d F4 F2 F20	2	10–24		25	26
Ld F6 0(R2)	3	4–4	8–12	14	27
Add.d F6 F4 F6	4	26–29		30	31
Sd F6 0(R2)	5	6–6	32–36		32
Addi R1 R1 -4	6	7–7		8	33
Addi R2 R2 -4	7	8–8		10	34

Instruction	ISSUE	EX	MEM	WB	COMMIT
Bne R1 R0 -12	8	9–9			35
Addi R1 R1 -4	10	11–11		12	36
Addi R2 R2 -4	11	12–12		13	37
Bne R1 R0 -12	12	13–13			38
Addi R1 R1 -4	14	15–15		16	39
Addi R2 R2 -4	15	16–16		17	40
Bne R1 R0 -12	16	17–17			41
Add.d F20 F2 F2	18	19–22		23	42

Issue width 2 (36 cycles)

Instruction	ISSUE	EX	MEM	WB	COMMIT
Ld F2 0(R1)	1	2–2	3–7	8	9
Mult.d F4 F2 F20	1	9–23		24	25
Ld F6 0(R2)	2	3–3	8–12	13	25
Add.d F6 F4 F6	2	25–28		29	30
Sd F6 0(R2)	3	4–4	31–35		31
Addi R1 R1 -4	3	4–4		5	32
Addi R2 R2 -4	4	5–5		6	32
Bne R1 R0 -12	4	6–6			33
Addi R1 R1 -4	7	8–8		9	33
Addi R2 R2 -4	7	9–9		10	34
Bne R1 R0 -12	8	10–10			34
Addi R1 R1 -4	11	12–12		13	35
Addi R2 R2 -4	11	13–13		14	35
Bne R1 R0 -12	12	14–14			36
Add.d F20 F2 F2	15	16–19		20	36

- With width 2, two instructions are issued per cycle and two commit per cycle (cycles 32, 33, 34 and 35).
- After each Bne nothing is issued until the cycle after its EX (issue 4, EX 6, next issue 7), because there is no branch prediction.
- The store starts writing memory in its commit cycle (31), after Add.d F6 commits in cycle 30.

Final values (all widths): R1=0, R2=20, F2=1.0, F4=3.0, F6=7.0, F20=2.0, Mem[32]=7.0. The other memory values do not change.

Test case 2: test2_independent

With 12 instructions that have few dependencies, the program goes from 21 cycles with issue width 1 to 16 with issue width 4.

Configuration (ROB entries: 64)

Unit	Reservation stations	Cycles in EX	Cycles in MEM
Integer adder	4	1	
FP adder	4	3	
FP multiplier	4	5	
Load/store unit	4	1	2

Initial values: R1=8, R2=16, R3=5, R4=7, F1=1.5, F2=2.5, F3=4.0; Mem[8]=10.0, Mem[16]=20.0

Program

```
Ld F4 0(R1)
Ld F5 0(R2)
Add.d F6 F1 F2
Mult.d F7 F2 F3
Add R5 R3 R4
Sub R6 R4 R3
Sub.d F8 F3 F1
Mult.d F9 F1 F1
Addi R7 R3 100
Add.d F10 F4 F5
Mult.d F11 F6 F7
Sd F10 8(R2)
```

Issue width 1 (21 cycles)

Instruction	ISSUE	EX	MEM	WB	COMMIT
Ld F4 0(R1)	1	2–2	3–4	5	6
Ld F5 0(R2)	2	3–3	5–6	9	10
Add.d F6 F1 F2	3	4–6		7	11
Mult.d F7 F2 F3	4	5–9		11	12
Add R5 R3 R4	5	6–6		8	13
Sub R6 R4 R3	6	7–7		10	14
Sub.d F8 F3 F1	7	8–10		12	15
Mult.d F9 F1 F1	8	9–13		15	16

Instruction	ISSUE	EX	MEM	WB	COMMIT
Addi R7 R3 100	9	10–10		13	17
Add.d F10 F4 F5	10	11–13		14	18
Mult.d F11 F6 F7	11	12–16		17	19
Sd F10 8(R2)	12	13–13	20–21		20

Issue width 4 (16 cycles)

Instruction	ISSUE	EX	MEM	WB	COMMIT
Ld F4 0(R1)	1	2–2	3–4	5	6
Ld F5 0(R2)	1	3–3	5–6	7	8
Add.d F6 F1 F2	1	2–4		5	8
Mult.d F7 F2 F3	1	2–6		7	8
Add R5 R3 R4	2	3–3		4	8
Sub R6 R4 R3	2	4–4		5	9
Sub.d F8 F3 F1	2	3–5		6	9
Mult.d F9 F1 F1	2	3–7		8	9
Addi R7 R3 100	3	5–5		6	9
Add.d F10 F4 F5	3	8–10		11	12
Mult.d F11 F6 F7	3	8–12		13	14
Sd F10 8(R2)	3	4–4	15–16		15

- With width 4 the 12 instructions are issued in 3 cycles instead of 12.
- The two loads issued in cycle 1 use the address adder in cycles 2 and 3. Add R5 and Sub R6, issued in cycle 2, use the integer adder in cycles 3 and 4: there is one unit of each type.
- Three results are broadcast in cycle 5 (Ld F4, Add.d F6 and Sub R6). Four instructions commit in cycle 8 and four in cycle 9.
- With width 4 the dependencies limit the program: Add.d F10 waits for the second load (WB in cycle 7), Mult.d F11 waits for Mult.d F7, and the store writes memory when it commits, after Mult.d F11.

Final values (all widths): R5=12, R6=2, R7=105, F4=10.0, F5=20.0, F6=4.0, F7=10.0, F8=2.5, F9=2.25, F10=30.0, F11=40.0, Mem[24]=30.0.

Test case 3: test3_forwarding

The program takes 50 cycles with issue width 1 and 48 with issue width 2. Store-to-load forwarding works, and the 3-entry load/store queue limits the gain. The configuration is the sample of the assignment text.

Configuration (ROB entries: 128)

Unit	Reservation stations	Cycles in EX	Cycles in MEM
Integer adder	2	1	
FP adder	3	3	
FP multiplier	2	20	
Load/store unit	3	1	4

Initial values: R1=10, R2=20, F2=30.1; Mem[4]=1, Mem[8]=2, Mem[12]=3.4, Mem[18]=7.5

Program

```
Add.d F1, F2, F3
Ld F4, 8(R1)
Mult.d F5, F4, F2
Sd F5, 0(R2)
Ld F6, 0(R2)
Sd F1, 4(R2)
Ld F7, -8(R2)
Ld F8, 4(R2)
Add.d F9, F6, F8
Sd F9, 2(R1)
Ld F10, 2(R1)
```

Issue width 1 (50 cycles)

Instruction	ISSUE	EX	MEM	WB	COMMIT
Add.d F1 F2 F3	1	2–4		5	6
Ld F4 8(R1)	2	3–3	4–7	8	9
Mult.d F5 F4 F2	3	9–28		29	30
Sd F5 0(R2)	4	5–5	31–34		31
Ld F6 0(R2)	5	6–6	30	31	32
Sd F1 4(R2)	6	7–7	35–38		35
Ld F7 -8(R2)	31	32–32	39–42	43	44
Ld F8 4(R2)	32	33–33	34	35	45
Add.d F9 F6 F8	33	36–38		39	46
Sd F9 2(R1)	35	36–36	47–50		47

Instruction	ISSUE	EX	MEM	WB	COMMIT
Ld F10 2(R1)	36	37–37	40	41	48

Issue width 2 (48 cycles)

Instruction	ISSUE	EX	MEM	WB	COMMIT
Add.d F1 F2 F3	1	2–4		5	6
Ld F4 8(R1)	1	2–2	3–6	7	8
Mult.d F5 F4 F2	2	8–27		28	29
Sd F5 0(R2)	2	3–3	30–33		30
Ld F6 0(R2)	3	4–4	29	30	31
Sd F1 4(R2)	3	5–5	34–37		34
Ld F7 -8(R2)	30	31–31	38–41	42	43
Ld F8 4(R2)	30	32–32	33	34	43
Add.d F9 F6 F8	31	35–37		38	44
Sd F9 2(R1)	34	35–35	45–48		45
Ld F10 2(R1)	34	36–36	39	40	46

- Ld F6 0(R2) gets its value from Sd F5 0(R2) by forwarding. It has one MEM cycle (29 with width 2), the cycle after the store data is broadcast (28). Ld F8 and Ld F10 are also forwarded from the store before them.
- Ld F7 -8(R2) has a different address from the stores before it, so it reads memory (4 cycles).
- With width 2, Ld F7 is issued in cycle 30 because the load/store queue (3 entries) is full until the first store commits.

Final values (all widths): F1=30.1, F4=7.5, F5=225.75, F6=225.75, F7=3.4, F8=30.1, F9=255.85, F10=255.85, Mem[12]=255.85, Mem[20]=225.75, Mem[24]=30.1.

Test case 4: test4_small_structures

With a ROB of 4 entries and 1 reservation station per unit, the program takes 48 cycles with issue width 1 and 46 with issue width 4: the small structures decide the timing.

Configuration (ROB entries: 4)

Unit	Reservation stations	Cycles in EX	Cycles in MEM
Integer adder	1	2	
FP adder	1	2	
FP multiplier	1	4	
Load/store unit	2	1	3

Initial values: R1=4, R2=3, F1=2.0, F2=3.0; Mem[4]=5.0, Mem[8]=6.0

Program

```
Addi R3 R1 4
Addi R4 R3 4
Add R5 R3 R4
Ld F3 0(R1)
Ld F4 0(R3)
Mult.d F5 F3 F4
Mult.d F6 F1 F2
Add.d F7 F5 F6
Add.d F7 F7 F1
Sub.d F8 F7 F2
Sd F8 0(R4)
Sd F7 4(R4)
Ld F9 0(R4)
Addi R0 R1 9
Add R6 R0 R2
Sub R10 R6 R5
Addi R11 R10 1
```

Issue width 1 (48 cycles)

Instruction	ISSUE	EX	MEM	WB	COMMIT
Addi R3 R1 4	1	2–3		4	5
Addi R4 R3 4	3	5–6		7	8
Add R5 R3 R4	6	8–9		10	11
Ld F3 0(R1)	7	8–8	9–11	12	13
Ld F4 0(R3)	8	9–9	12–14	15	16
Mult.d F5 F3 F4	9	16–19		20	21
Mult.d F6 F1 F2	17	18–21		22	23
Add.d F7 F5 F6	18	23–24		25	26
Add.d F7 F7 F1	24	26–27		28	29
Sub.d F8 F7 F2	27	29–30		31	32
Sd F8 0(R4)	28	29–29	33–35		33
Sd F7 4(R4)	29	30–30	36–38		36
Ld F9 0(R4)	33	34–34	39–41	43	44
Addi R0 R1 9	34	35–36		37	45
Add R6 R0 R2	36	37–38		39	46
Sub R10 R6 R5	38	40–41		42	47
Addi R11 R10 1	44	45–46		47	48

Issue width 4 (46 cycles)

Instruction	ISSUE	EX	MEM	WB	COMMIT
Addi R3 R1 4	1	2–3		4	5
Addi R4 R3 4	3	5–6		7	8
Add R5 R3 R4	6	8–9		10	11
Ld F3 0(R1)	6	7–7	8–10	11	12
Ld F4 0(R3)	6	8–8	11–13	14	15
Mult.d F5 F3 F4	8	15–18		19	20
Mult.d F6 F1 F2	16	17–20		21	22
Add.d F7 F5 F6	16	22–23		24	25
Add.d F7 F7 F1	23	25–26		27	28
Sub.d F8 F7 F2	26	28–29		30	31
Sd F8 0(R4)	26	27–27	32–34		32
Sd F7 4(R4)	26	28–28	35–37		35
Ld F9 0(R4)	32	33–33	38–40	41	42
Addi R0 R1 9	32	33–34		35	42
Add R6 R0 R2	34	35–36		37	42
Sub R10 R6 R5	36	38–39		40	42
Addi R11 R10 1	42	43–44		45	46

- With width 4, only one instruction is issued in cycle 1: the second Addi needs the only reservation station of the integer adder, so the issue stops there.
- In cycle 6 three instructions are issued and the fourth, Mult.d F5, is not: the ROB is full. It is issued in cycle 8, when Addi R4 commits.
- Mult.d F6 is issued in cycle 16, after Mult.d F5 leaves the only reservation station of the multiplier (EX starts in cycle 15).
- F7 is written by two instructions. Add.d F7 F7 F1 reads the result of the first one and Sub.d F8 F7 F2 reads the result of the second one, so F7=38.0 and F8=35.0.
- Addi R0 R1 9 does not change R0: Add R6 R0 R2 gives R6=3.
- Four instructions commit in cycle 42.

Final values (all widths): R3=8, R4=12, R5=20, R6=3, R10=-17, R11=-16, F3=5.0, F4=6.0, F5=30.0, F6=6.0, F7=38.0, F8=35.0, F9=35.0, Mem[12]=35.0, Mem[16]=38.0.

Test case 5: test5_branches

The program takes 69 cycles with issue width 1 and 72 with issue width 2: with branches and a 4-entry load/store queue, a wider issue is slightly slower. The input is written in lower case with commas.

Configuration (ROB entries: 16)

Unit	Reservation stations	Cycles in EX	Cycles in MEM
Integer adder	3	1	
FP adder	3	2	
FP multiplier	2	6	
Load/store unit	4	1	2

Initial values: r1=24, r2=4, r3=40, f10=0.5; mem[4]=1.0, mem[8]=2.0, mem[12]=3.0, mem[16]=4.0, mem[20]=5.0, mem[24]=6.0

Program

```
beq R1, R2, 20
ld F1, 0(R1)
mult.d F2, F1, F10
add.d F3, F3, F2
sd F3, 0(R3)
ld F4, 0(R3)
SD F4, 40(R1)
ADDI R1, R1, -4
BNE R1, R2, -32
beq R0, R0, 4
addi R20, R0, 99
add.d F20, F3, F4
```

Issue width 1 (69 cycles)

Instruction	ISSUE	EX	MEM	WB	COMMIT
Beq R1 R2 20	1	2–2			3
Ld F1 0(R1)	3	4–4	5–6	7	8
Mult.d F2 F1 F10	4	8–13		14	15
Add.d F3 F3 F2	5	15–16		17	18
Sd F3 0(R3)	6	7–7	19–20		19
Ld F4 0(R3)	7	8–8	18	19	20
Sd F4 40(R1)	8	9–9	21–22		21
Addi R1 R1 -4	9	10–10		11	22
Bne R1 R2 -32	10	12–12			23

Instruction	ISSUE	EX	MEM	WB	COMMIT
Ld F1 0(R1)	13	14–14	15–16	18	24
Mult.d F2 F1 F10	14	19–24		25	26
Add.d F3 F3 F2	15	26–27		28	29
Sd F3 0(R3)	16	17–17	30–31		30
Ld F4 0(R3)	19	20–20	29	30	31
Sd F4 40(R1)	20	21–21	32–33		32
Addi R1 R1 -4	21	22–22		23	33
Bne R1 R2 -32	22	24–24			34
Ld F1 0(R1)	25	26–26	27–28	29	35
Mult.d F2 F1 F10	26	30–35		36	37
Add.d F3 F3 F2	27	37–38		39	40
Sd F3 0(R3)	28	29–29	41–42		41
Ld F4 0(R3)	30	31–31	40	41	42
Sd F4 40(R1)	31	32–32	43–44		43
Addi R1 R1 -4	32	33–33		34	44
Bne R1 R2 -32	33	35–35			45
Ld F1 0(R1)	36	37–37	38–39	40	46
Mult.d F2 F1 F10	37	41–46		47	48
Add.d F3 F3 F2	38	48–49		50	51
Sd F3 0(R3)	39	40–40	52–53		52
Ld F4 0(R3)	41	42–42	51	52	53
Sd F4 40(R1)	42	43–43	54–55		54
Addi R1 R1 -4	43	44–44		45	55
Bne R1 R2 -32	44	46–46			56
Ld F1 0(R1)	47	48–48	49–50	51	57
Mult.d F2 F1 F10	48	52–57		58	59
Add.d F3 F3 F2	49	59–60		61	62
Sd F3 0(R3)	50	51–51	63–64		63
Ld F4 0(R3)	52	53–53	62	63	64
Sd F4 40(R1)	53	54–54	65–66		65
Addi R1 R1 -4	54	55–55		56	66
Bne R1 R2 -32	55	57–57			67

Instruction	ISSUE	EX	MEM	WB	COMMIT
Beq R0 R0 4	58	59–59			68
Add.d F20 F3 F4	60	64–65		66	69

Issue width 2 (72 cycles)

Instruction	ISSUE	EX	MEM	WB	COMMIT
Beq R1 R2 20	1	2–2			3
Ld F1 0(R1)	3	4–4	5–6	7	8
Mult.d F2 F1 F10	3	8–13		14	15
Add.d F3 F3 F2	4	15–16		17	18
Sd F3 0(R3)	4	5–5	19–20		19
Ld F4 0(R3)	5	6–6	18	19	20
Sd F4 40(R1)	5	7–7	21–22		21
Addi R1 R1 -4	6	7–7		8	22
Bne R1 R2 -32	6	9–9			22
Ld F1 0(R1)	10	11–11	12–13	14	23
Mult.d F2 F1 F10	10	15–20		21	23
Add.d F3 F3 F2	11	22–23		24	25
Sd F3 0(R3)	12	13–13	26–27		26
Ld F4 0(R3)	19	20–20	25	26	27
Sd F4 40(R1)	19	21–21	28–29		28
Addi R1 R1 -4	20	21–21		22	29
Bne R1 R2 -32	20	23–23			29
Ld F1 0(R1)	24	25–25	30–31	32	33
Mult.d F2 F1 F10	24	33–38		39	40
Add.d F3 F3 F2	25	40–41		42	43
Sd F3 0(R3)	26	27–27	44–45		44
Ld F4 0(R3)	26	28–28	43	44	45
Sd F4 40(R1)	28	29–29	46–47		46
Addi R1 R1 -4	28	29–29		30	47
Bne R1 R2 -32	29	31–31			47
Ld F1 0(R1)	32	33–33	34–35	36	48
Mult.d F2 F1 F10	32	37–42		43	48
Add.d F3 F3 F2	33	44–45		46	49

Instruction	ISSUE	EX	MEM	WB	COMMIT
Sd F3 0(R3)	34	35–35	50–51		50
Ld F4 0(R3)	44	45–45	47	48	51
Sd F4 40(R1)	44	46–46	52–53		52
Addi R1 R1 -4	45	46–46		47	53
Bne R1 R2 -32	45	48–48			53
Ld F1 0(R1)	49	50–50	54–55	56	57
Mult.d F2 F1 F10	49	57–62		63	64
Add.d F3 F3 F2	50	64–65		66	67
Sd F3 0(R3)	50	51–51	68–69		68
Ld F4 0(R3)	51	52–52	67	68	69
Sd F4 40(R1)	52	53–53	70–71		70
Addi R1 R1 -4	52	53–53		54	71
Bne R1 R2 -32	53	55–55			71
Beq R0 R0 4	56	57–57			72
Add.d F20 F3 F4	58	69–70		71	72

- The first Beq is not taken (R1 is 24 and R2 is 4). The loop body runs 5 times: Bne is taken 4 times and not taken the last time. Beq R0 R0 4 is taken and skips addi R20, R0, 99, so R20 stays 0 and that instruction is not in the tables.
- In every iteration Ld F4 0(R3) gets its value from Sd F3 0(R3) by forwarding (one MEM cycle).
- Width 2 is slower than width 1. The results are correct (they are equal to the reference model); the cause is contention. With a wider issue, the stores of one iteration enter the load/store queue earlier and stay there until they commit, so the queue is full and the issue stalls: Ld F4 0(R3) of the second iteration is issued in cycle 19, 7 cycles after the store before it.
- The first load of the third iteration (EX in cycle 25) also waits for the memory port until cycle 30, behind two stores that are committing.
- With a load/store queue of 6 entries the same program takes 65, 60, 59 and 59 cycles for issue width 1 to 4.

Final values (all widths): R1=4, R20=0, F1=2.0, F2=1.0, F3=10.0, F4=10.0, F20=20.0, Mem[40]=10.0, Mem[48]=10.0, Mem[52]=9.0, Mem[56]=7.5, Mem[60]=5.5, Mem[64]=3.0.

Test case 6: test6_class_example

The simulator gives exactly the timing tables of the course lecture slides: 40 cycles with issue width 1 and 39 with issue width 2. The example is worked cycle by cycle in

CompArch-OOO-Example.pdf (slides 9 to 40 with one issue per cycle, slides 41 to 63 with two).

Configuration (ROB entries: 7)

Unit	Reservation stations	Cycles in EX	Cycles in MEM
Integer adder	1	1	
FP adder	2	4	
FP multiplier	2	10	
Load/store unit	2	1	6

Initial values: R1=100, R2=200, F4=2.5; Mem[124]=10.0, Mem[232]=0.5

Program

Ld F6, 24(R1)
Ld F2, 32(R2)
Mult.d F1, F2, F4
Mult.d F8, F6, F1
Add.d F10, F1, F2
Addi R2, R2, 4

Issue width 1 (40 cycles)

Instruction	ISSUE	EX	MEM	WB	COMMIT
Ld F6 24(R1)	1	2–2	3–8	9	10
Ld F2 32(R2)	2	3–3	9–14	15	16
Mult.d F1 F2 F4	3	16–25		26	27
Mult.d F8 F6 F1	4	27–36		37	38
Add.d F10 F1 F2	5	27–30		31	39
Addi R2 R2 4	6	7–7		8	40

Issue width 2 (39 cycles)

Instruction	ISSUE	EX	MEM	WB	COMMIT
Ld F6 24(R1)	1	2–2	3–8	9	10
Ld F2 32(R2)	1	3–3	9–14	15	16
Mult.d F1 F2 F4	2	16–25		26	27
Mult.d F8 F6 F1	2	27–36		37	38
Add.d F10 F1 F2	3	27–30		31	38
Addi R2 R2 4	3	4–4		5	39

- All the cycles are the same as in the slides (final tables on slide 35 and slide 63). run_tests.py compares the two tables automatically.
- With width 2, Mult.d F8 and Add.d F10 commit in the same cycle (38).
- Mult.d F1 and Mult.d F8 are issued in the same cycle, and the second one waits for the result of the first one (EX from cycle 27, after the broadcast in cycle 26).
- The slides assume FP units that are not pipelined, and the simulator has pipelined FP units, as the assignment says. In this program the result is the same, because the second multiply needs the result of the first one.

Final values (all widths): R2=204, F1=1.25, F2=0.5, F6=10.0, F8=12.5, F10=1.75.

How to run

unzip tomasulo-algorithm.zip

(or: git clone https://github.com/dprol/tomasulo-algorithm.git)

cd tomasulo-algorithm/code

python3 main.py # provided test case, issue width 1

python3 main.py ../tests/test2_independent.txt 4 # input file, issue width

python3 run_tests.py # all tests, issue width 1 to 4

Python 3 is required; no other package is needed.